Research

I will need to research the different ways of making the physical calculator, such as what the main board should be, how to connect it to some sort of screen, and also learn how to make the underlying PCB. I will then need to find out how to write the software for it, which will need to compile to run on both a computer and on the physical device as well.

Hardware Comparison

For the main controller, I have 2 options: either a small Linux computer, or a microcontroller. Both of these have their own pros and cons which I will explore in this section.

Microcontroller:

Advantages	Disadvantages
• Would have a lower power draw, making it run for longer off a small battery/cell. • The boot time would be almost instant, because there is no OS to load • A microcontroller would also be a lot cheaper to buy than a Linux single board computer.	• The processing power in a microcontroller is not that good, and complex functions or graphs may take some time to compute. • The onboard memory in most microcontrollers is very limited, up to a few megabytes in most cases. This means I would need to take extra care in my programming • The lower computing power would also mean I would need to heavily optimize my code to run efficiently on the microcontroller.

Linux computer:

Advantages	Disadvantages
• Would have higher processing power than a microcontroller, meaning that equations and functions can be solved quicker • Easier interface with a display, can use interfaces such as HDMI, VGA for video signals	• The price is usually a big higher for a single board computer, averaging at around £20 for lower end options, compared to around £5 for a microcontroller • It may also take a lot longer than a microcontroller to start up since the computer would need to load an OS before it can run any programs

	• The battery life would be a lot worse on a single board computer since it is a lot less efficient to be running a full operating system in the background

The technical research aspect of this project can be further broken down into the following topics which are the different features of the calculator:

- Evaluating expressions
- Misc. operations
- Statistics
    - Binomial probability and cumulative distributions
    - Normal cumulative and inverse normal distributions
    - Poisson probability and cumulative distributions
- Graph plotting
- Equation solving
    - Solve unknown
    - Simultaneous Equations
    - Polynomials
- Numerical Methods
    - Solutions of equations
    - Curve fitting/regression
    - Numerical integration and differentiation
- UI Libraries

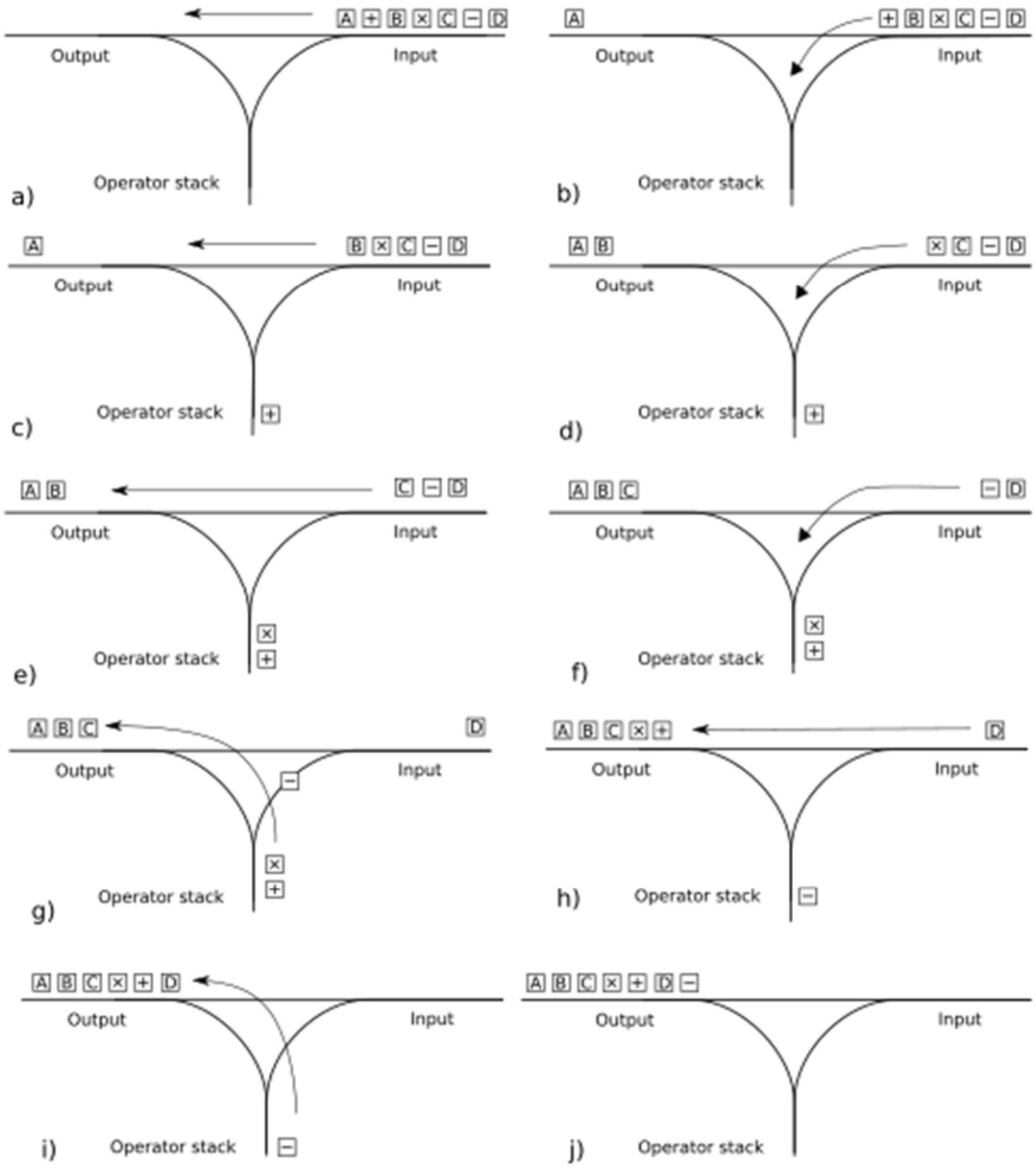
Evaluating Expressions

For the main calculator part of the program, the computer needs to know how to evaluate an expression given in a string.

There are a few different ways I can parse the expression string into something that can be solved. There is no simple `eval()` function in C++ so I will have to make my own. The methods of this would be in a separate `parser` file. For example, a very basic left-to-right parser, where it simply scans across the string, performing operations are quite straightforward:

20 + 30 * 5 + 10
20+30 = 50
50*5 = 250
250+10 = 260

This is an extremely basic example of how the program could parse through a string and evaluate it. There are quite a few drawbacks by using this however, for example the expression is evaluated as-is from left to right, meaning that multiplication would not take priority over addition or subtraction. For example, `2+2*4` would return `16` and not `10` like it should. One example of how we can combat this is by using Dijkstra's Shunting yard algorithm, which takes an expression in infix notation (the 'standard' mathematical notation with operators in between numbers, e.g. 2+4), and parses it into a postfix notation string (where operators follow their operands, e.g. 2 4 +), which makes it easier to follow mathematical rules of operation orders.



For example, if we want to evaluate the expression $1 + 2 \times 3 - 4$, the procedure would be as follows:

Converting to postfix notation

Tokens	Token	Output	Operator Stack	Notes
['1', '+', '2', '×', '3', '-', '4']		[]	[]	Numbers and operators separated into array elements
['+', '2', '×', '3', '-', '4']	'1'	['1']	[]	First token is integer so push to output
['2', '×', '3', '-', '4']	'+'	['1']	['+']	Next token is an operator so push to operator stack
['×', '3', '-', '4']	'2'	['1', '2']	['+']	Integer pushed to output
['3', '-', '4']	'×'	['1', '2']	['+', '×']	'×' has a higher priority than '+' so push to operator stack
['-', '4']	'3'	['1', '2', '3']	['+', '×']	Integer pushed to output
['4']	'-'	['1', '2', '3', '×']	['+']	'-' has a lower priority than '×', so '×' is popped from the operator stack and pushed to the output
['4']	'-'	['1', '2', '3', '×', '+']	[]	'-' has equal priority to '+', so '+' is popped from the operator stack and pushed to the output
['4']	'-'	['1', '2', '3', '×', '+']	['-']	'-' is then added to the operator stack
[]	'-'	['1', '2', '3', '×', '+', '4']	['-']	Integer pushed to output
[]	'4'	['1', '2', '3', '×', '+', '4', '-']	[]	Remaining operators ('-') pushed to output one by one since there are no more input tokens.

This will leave us with 1 2 3 × + 4 −, which is easier to evaluate than what we had before.

Evaluating the postfix:

Tokens	Token	Output	Notes
['1', '2', '3', '×', '+', '4', '-']		[]	
['2', '3', '×', '+', '4', '-']	'1'	['1']	'1' pushed to output
['3', '×', '+', '4', '-']	'2'	['1', '2']	'2' pushed to output
['×', '+', '4', '-']	'3'	['1', '2', '3']	'3' pushed to output
['+', '4', '-']	'×'	['1', '6']	'×' is an operator, so pop last 2 elements of output, and compute them with the operator. So here, compute 2×3 which is 6
['4', '-']	'+'	['7']	'+' is an operator, so pop last 2 elements and compute with operator: 6 + 1 = 7.
['-']	'4'	['7', '4']	'4' pushed to output
[]	'-'	['3']	'-' is an operator, so compute 7 – 4 which is 3

Which results in an output of 3, which is the correct solution to 1 + 2 × 3 – 4

Another alternative is the recursive descent parsing algorithm. Instead of converting the expression into a different notation, recursive descent works directly on the original infix notation by breaking it down into a series of recursive functions that preserve operator precedence.

There are separate functions for the different parts of the expression, such as one for addition/subtraction, one for multiplication/division, and one for handling numbers and brackets. An expression like 2 + 2 * 4 would therefore be evaluated as 2 + (2 x 4) without needing an extra conversion step.

One of the main advantages of recursive descent is code clarity. The code retains the original formatting of the expression and is similar to the mathematical language as well, making it easy to read and debug. Adding more functions such as sin(x) is more straightforward, as you can simply add another parsing function to the parser.

Parsing and evaluating using recursive descent:

Function Call	Remaining Tokens	Operation	Result
expression()	1 + 2 × 3 − 4	parse first term	
term()	1 + 2 × 3 − 4	parse factor	1
expression()	+ 2 × 3 − 4	+ found	
term()	2 × 3 − 4	parse factor	2
term()	× 3 − 4	× found	
factor()	3 − 4	multiply	6
expression()	− 4	add	1 + 6 = 7
expression()	− 4	- found	
term()	4	parse factor	4
expression()		subtract	7 − 4 = 3

Statistics

The Binomial Distribution

The binomial distribution is a way of calculating probabilities of a random event occurring n times when the probability of success is known. One such example of when this may be useful is calculating the probabilities of different outcomes when flipping a number of coins.

For example, when flipping 2 coins, there are 4 possible outcomes:

[Heads, Heads]

[Heads, Tails]

[Tails, Heads]

[Tails, Tails]

Here, if we wanted to work out the probability of getting 2 heads, the process is relatively simple. Using the formula $\frac{Number\ of\ favourable\ outcomes}{Total\ number\ of\ possible\ outcomes}$, so the probability would be $\frac{1}{4}$ or 0.25. This could also be done by using the formula $P(A \cap B) = P(A) \times P(B)$, where A is the first coin and B is the second coin. However, things aren't so simple when you want to work out the probability of a different combination of events, such as 1 heads and 1 tails.

Since there are 2 possible ways of getting 1 heads and 1 tails ([Heads, Tails] and [Tails, Heads]), you would need to use the binomial distribution formula:

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$$

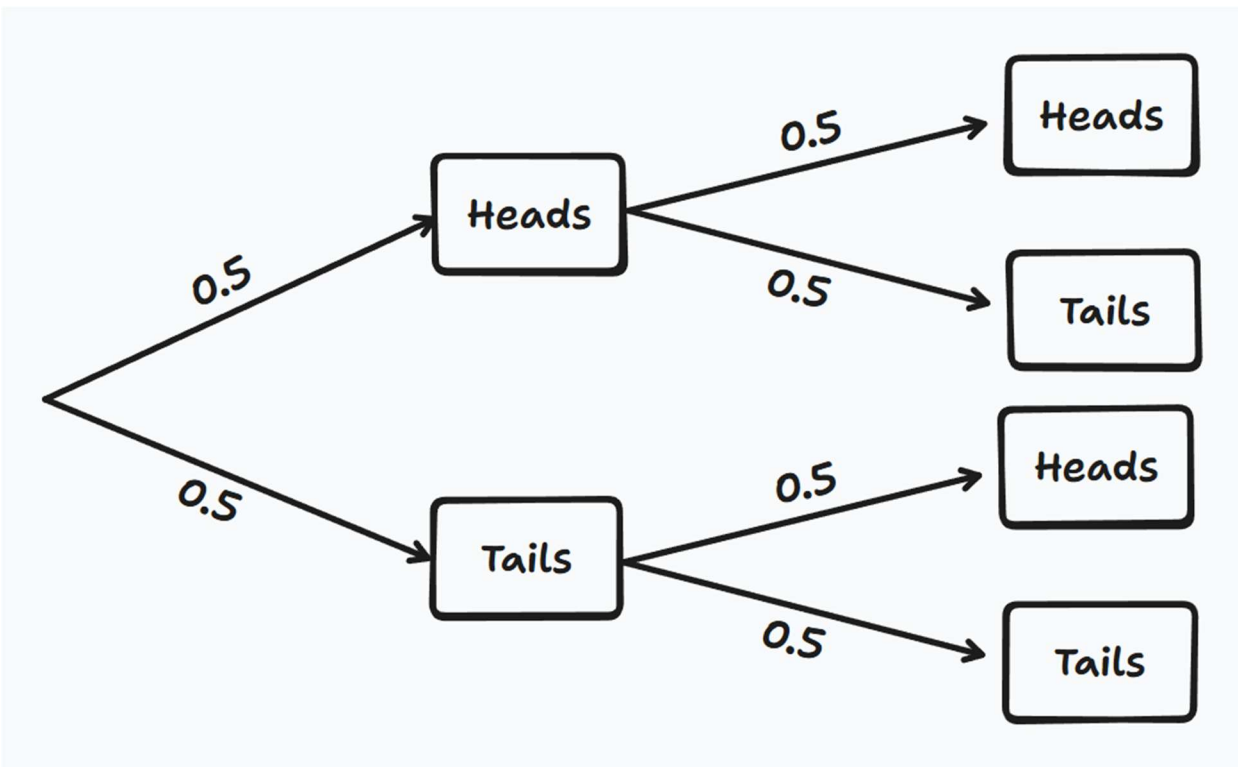
Where $\binom{n}{k}$ is the number of ways to choose k successes from n outcomes, calculated by the formula

$$\frac{n!}{k!\,(n-k)!}$$

Using the formula, we can work out the probability of any combination of events occurring in n trials, with k successes and p probability of success in one trial. Going back to the example, the probability of getting 1 heads and 1 tails when flipping a coin 2 times would be calculated using the formula, with a 0.5 probability of success, 2 trials and 1 successful trial:

$$P(X=1) = \binom{2}{1} 0.5^1 (1-0.5)^{2-1} = 0.5$$

This can be proven true as shown by the following probability tree diagram:



As shown above, the probability of getting [Heads, Tails] is (0.5*0.5) = 0.25, and the same is true for getting [Tails, Heads]. Combining these two results by adding them (union of 2 probabilities) and you get 0.5, the same as the result shown by the binomial formula.

The binomial distribution formula might look a bit unnecessarily complex, however this quickly scales up as the number of trials increases. For example, if a coin was flipped 100 times, then working out each combination of getting a certain number of heads would take an overly long time to calculate and is made a lot easier by using the formula.

Binomial Cumulative Distribution

Similar to the Binomial probability distribution mentioned above, the binomial cumulative distribution is a way of calculating the probability of getting up to a certain number of successes in a fixed number of trials, in the form $P(X \leq x)$. This distribution is similar to the previous one, except it adds all of the probabilities from 0 – x to work it out. For example, if trying to work out $P(X \leq 3)$, the calculator would do $P(X = 3) + P(X = 2) + P(X = 1) + P(X = 0)$. In the previous section we found out how to calculate these kinds of probabilities and the binomial cumulative distribution is simply the sum of many such probabilities. As a result, the general formula of this distribution can be shown using the formula

$$P(X \leq k) = \sum_{k=0}^{x} \binom{n}{k} p^k (1-p)^{n-k}$$

Normal Cumulative Distribution

The normal distribution is a way of modelling continuous data that tends to cluster around an average value. Things like height, weight or exam scores. The normal distribution takes the shape of a bell curve, the highest point being in the middle and a symmetric falloff on both sides.

The normal distribution takes 2 values:

- The mean (μ) which represents the highest point on the graph
- The standard distribution σ, which is a measure of how spread out the data is

If the standard distribution is small, then more of the data is close to the mean, and if it is large, then the curve is flatter and more spread out.

Generally the following rules are a good approximation of the percentage of data that is 1, 2 or 3 standard deviations away from the mean:

- Within 1σ (μ ± 1σ): about 68% of the data
- Within 2σ (μ ± 2σ): about 95% of the data
- Within 3σ (μ ± 3σ): about 99.7% of the data

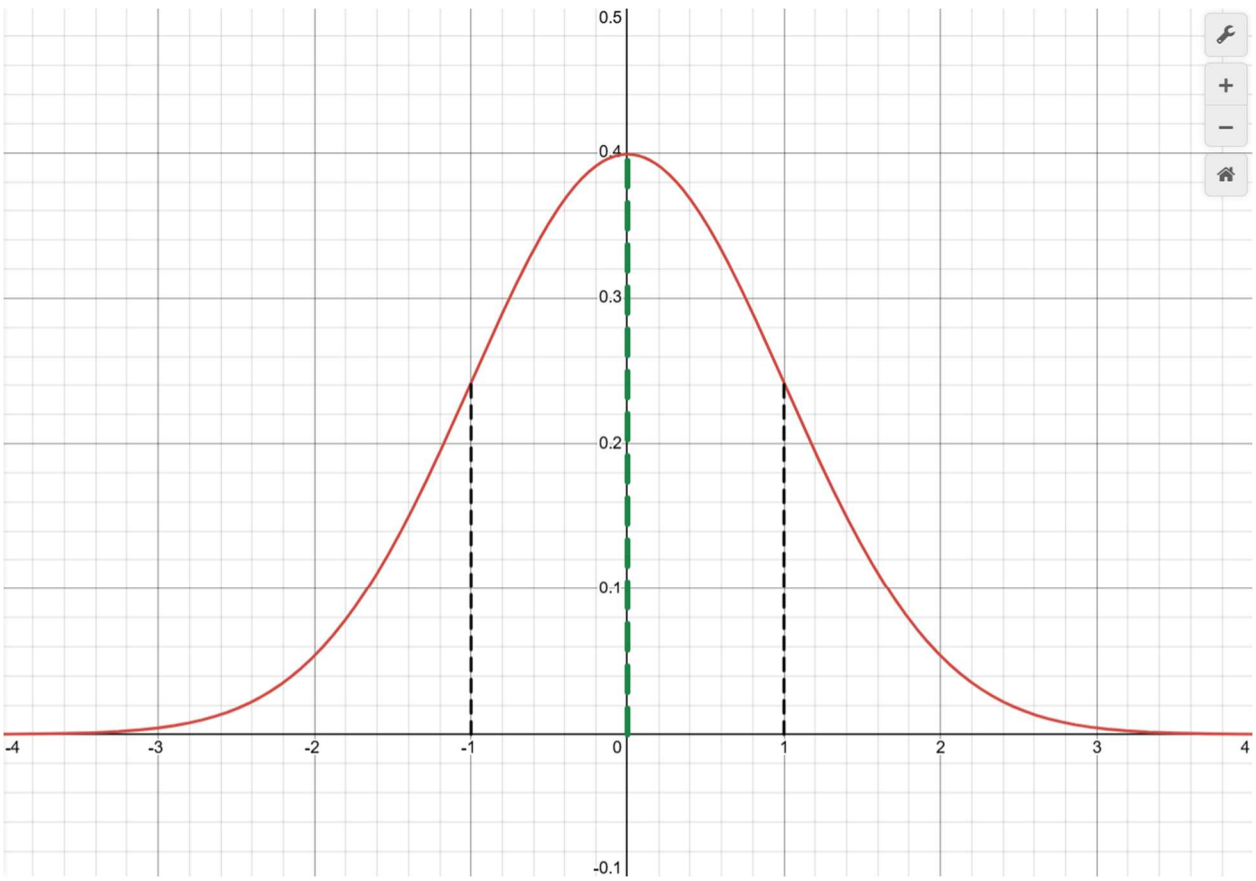
Since the normal distribution uses continuous data rather than discrete, the probability of getting a specific value (e.g. 175cm) is extremely low. Instead, the probability is calculated over a range of values, such as between 170 and 180cm.

To calculate these probabilities, we use the cumulative distribution function. This gives us the probability that X is less than or equal to a certain value. The formula for his is as follows:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

To work out the probability of a result in between 2 values, this curve is integrated between the two:

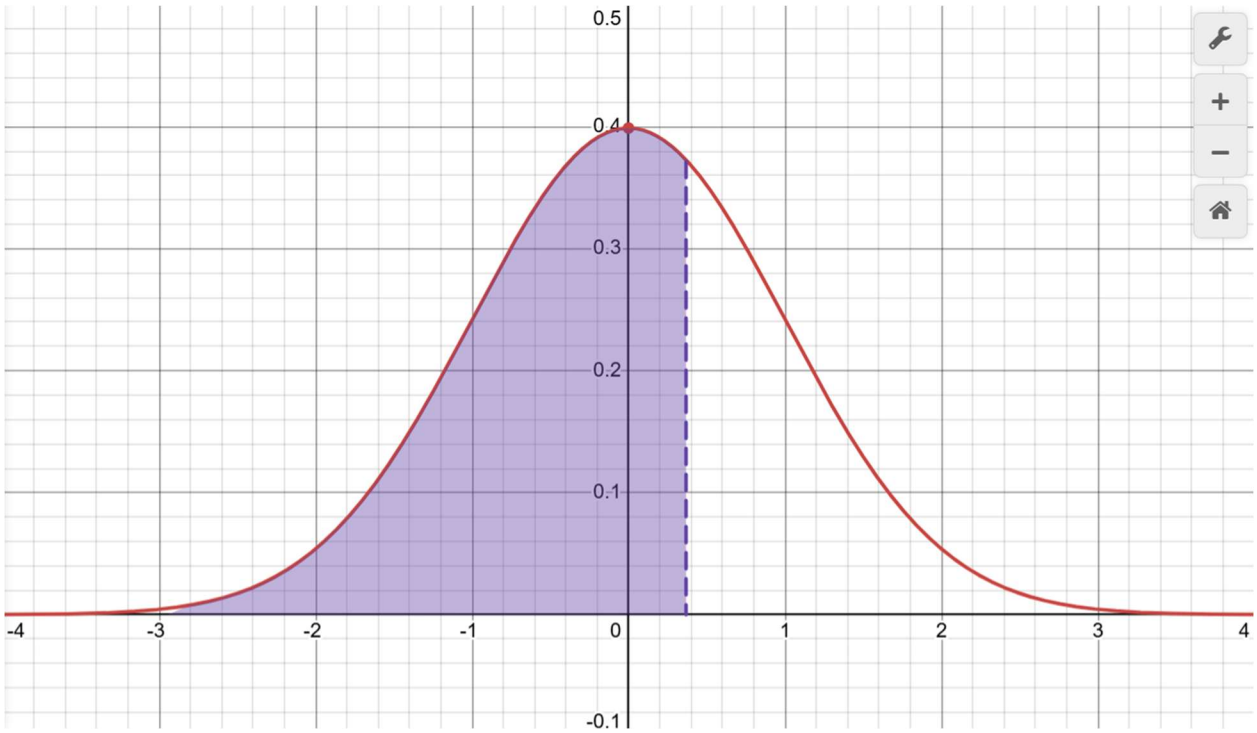
$$P(a \leq X \leq b) = \int_a^b \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$



Inverse normal distribution

The inverse normal distribution is used when we have a probability and are trying to work out the value that corresponds to it. Instead of calculating the probability of a value lying within a range, the inverse normal distribution finds the point on the curve where a known

percentage of data lies below it.
For example, this can be used to find percentiles, such as exam grade boundaries. The inverse normal distribution can be used to find the score that corresponds to the top 10% of test takers.



Calculating this requires use of the Gauss inverse error function, however this is very difficult to compute as it involves complex mathematics and has no simple algebraic implementation. Instead, we can use a polynomial approximation of this function called the Peter John Acklam's approximation

Given the probability p, the mean μ, and the standard deviation σ, the value of x such that $P(X \leq x) = p$ can be worked out by first reducing the problem to a standard normal distribution with mean 0 and standard deviation 1.

If $p < 0.5$, $t = \sqrt{-2\ln(p)}$.

If $p > 0.5$, $t = \sqrt{-2\ln(1-p)}$.

This converts the probability into a number that can be approximated accurately. Now the value of z can be approximated using the fraction:

$$z = t - \frac{c_0 + c_1 t + c_2 t^2}{1 + d_1 t + d_2 t^2 + d_3 t^3}$$

The values for c_i and d_i are the standard Acklam coefficients and are listed below:

$$c_0 = 2.515517$$

$$c_1 = 0.802853$$

$$c_2 = 0.010328$$

$$d_1 = 1.432788$$

$$d_2 = 0.189269$$

$$d_3 = 0.001308$$

If $p < 0.5$, use $z = -z$.

Finally, scale back using the formula

$$x = \mu + \sigma z$$

to give the final x value for which the probability is p.

Graph Plotting

Equation Solving

Numerical Methods

UI Libraries

When making the UI of the calculator, I have to pick a suitable UI library that will allow me to compile the software to run on both PC and on the physical calculator. Here are the options I found:

UI Library	Details
µGFX	- Good low level control for rendering - Stable software so unlikely to stop working - Less friendly API when building more complex layouts - No desktop simulator meaning code cannot be tested on PC.
Adafruit GFX	- Very lightweight and fast - Easy to draw text and graphics to displays - Simple to use with a microcontroller - No built-in UI components or layout system - Complex interfaces must be implemented manually - No desktop simulator for testing on PC

LVGL	- Proper UI components such as buttons, labels, etc. - Designed specifically for embedded systems - Easy to use styling and layout system - Well documented usage - Desktop simulator available meaning code can compile to run on windows - Memory usage is slightly higher than other low-level alternatives
Qt	- Very polished and professional UI framework - Powerful layout and animation system - Windows support - High memory/storage requirements - Complex build system
Custom UI	- Full control over how components are rendered - No abstraction - Requires a lot more development effort - Easy to make bugs/inconsistent UI - Nothing built in meaning everything needs to programmed from scratch.

Existing Solutions

Casio FX-CG50

The Casio FX-CG50 is a high-end graphing calculator usually used in colleges and sixth forms for A-level further mathematics or similar courses. It has a variety of features such as:

- Matrices
- Statistics
- Spreadsheet
- 2D and 3D Graph plotting

Pros	• Has a wide range of functions • Allowed for use in examinations • Does not require an internet connection • Physical buttons which may be a better user experience